EXPERIMENT NO: 7

Student Name: Ayush kohli **UID:** 23BCS11238

Branch: BE-CSE **Section/Group:** KRG-3B

Semester: 6th **Date of Performance:** 01/04/2026

Subject Name: System Design **Subject Code:** 23CSH-314

Aim

Develop a cab-booking application similar to Ola or Uber.

Objectives

- Design a scalable, real-time messaging system similar to Ola/Uber
- Enable instant communication between drivers in a given area
- Support both text and voice communication
- Ensure high availability and low-latency messaging
- Maintain strong privacy with end-to-end encryption
- Efficiently handle billions of messages per day
- Provide a seamless user experience across devices
- Support horizontal scaling as the user base grows
- Ensure reliable message delivery and synchronization

Procedure

1. Requirement Analysis

functional and non-functional requirements
Define scale assumptions (e.g., number of users, messages per second)

2. System Architecture Design

- Choose a distributed architecture
- Define core services:
    - Authentication Service
    - Messaging Service
    - Media Service
    - Notification Service
    - Presence Service

3. Communication Protocol Design

- Use WebSockets for real-time messaging
- Use HTTP/HTTPS for APIs

4. Data Storage Design

- User database
- Message database
- Media object storage

- Cache layer (e.g., Redis)

5. Scalability Planning

- Implement load balancers
- Define a sharding strategy
- Enable horizontal scaling

6. Reliability Design

- Use message queues
- Implement retry mechanisms
- Enable data replication

7. Security Design

- Apply encryption protocols
- Use secure authentication tokens
- Implement a key exchange system

8. Deployment Design

- Use cloud infrastructure
- Integrate CDN for media delivery
- Set up monitoring tools

Functional Requirements

User Management

1. User registration via phone number
2. OTP verification and authentication
3. Profile creation and updates
4. Contact synchronization
5. User presence status (online/offline/last seen)

Communication Features

- Voice calls
- Push notifications for new messages

Multi-Device Support

- Synchronization of messages across multiple devices

Non-Functional Requirements

Performance

- Message delivery latency: 100–300 ms
- Support millions of concurrent users
- Fast media upload and download

Scalability

- Horizontal scaling of servers
- Distributed architecture

- Load balancing

Availability
- Target uptime of 99.99%
- Fault-tolerant services
- Automatic failover mechanisms

Reliability
- Guaranteed message delivery
- Zero message loss
- Retry mechanisms

Security
- End-to-end encryption
- Secure key management
- Compliance with data privacy regulations

Consistency
- Message ordering within each conversation
- Eventual consistency across devices

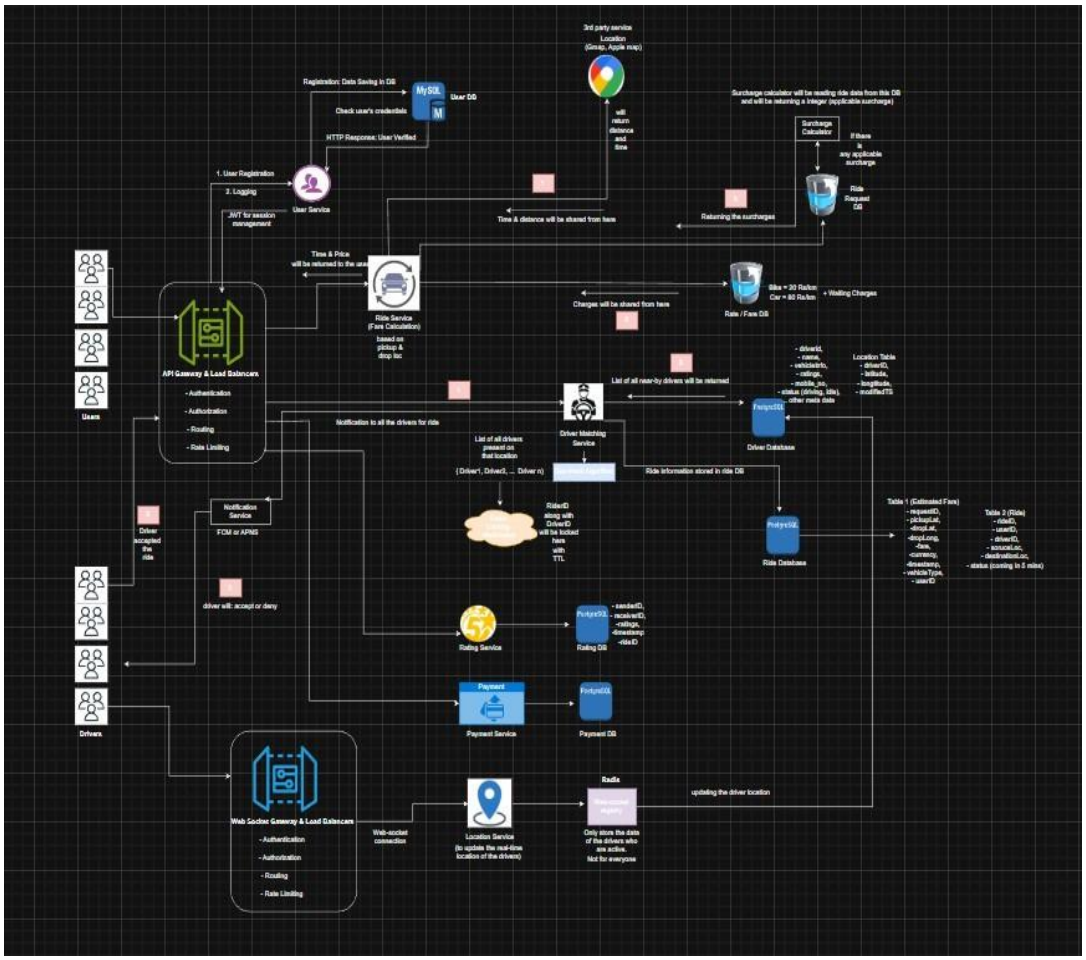
Maintainability
- Modular microservices architecture
- Easy deployment and updates

Observability
- Logging
- Monitoring
- Alerting systems

Storage Efficiency
- Efficient database indexing
- Separate storage for media (object storage)

High-Level Design (HLD)

The system consists of client applications, an API gateway, and multiple services such as User Service, Subscription Service, Search Service, Video Streaming Service, and a Recommendation Engine. Supporting components include CDN, cache, message queues, and distributed databases. Video content is stored in object storage and delivered via CDN to ensure low-latency streaming.

Outcome

- Understand large-scale distributed system architecture
- Design real-time messaging systems
- Apply scalability and load-balancing concepts
- Understand microservices architecture
- Implement message queues and event-driven systems
- Design secure communication systems
- Analyze trade-offs between consistency and availability
- Learn database partitioning and replication
- Understand real-time protocols (WebSockets)
- Apply industry-standard system design best practices